

Name: _____

Roll number: _____

Time Allowed: 15 min

Total Marks: 20

Question#1 (3+3+4=10 Marks)

----- CLO3

Write down the output of the following codes:

<pre>int main() { int arr[5] = {65, 67, 68}; int *ptr1 = arr; cout << *(ptr1 + 2) << " " << *(ptr1 + 0) << endl; return 0; }</pre>	68 65
<pre>int main(){ int x = 70, y = 80, z = 90; int * ptr; cout << x << " " << y << " " << z << endl; ptr = &x; *ptr *= 10; ptr = &y; *ptr += 5; ptr = &z; *ptr *= 2; cout << x << " " << y << " " << z << endl; return 0; }</pre>	70 80 90 700 85 180
<pre>void inc_count(int *count_ptr) { (*count_ptr)++; } int main() { int count = 0; while (count < 10) inc_count(&count); cout << count << endl; return 0; }</pre>	10

Question#2 (10 Marks)

----- CLO3

Considering the following code:

<pre>struct Item { char name[50]; int quantity; float unitPrice; };</pre>	<pre>struct Cart { Item * data; int noOfItems; int capacity; };</pre>
---	---

Write down the definition of the following function, it will Increase the capacity of the Cart by s (default value is 5). Ensure that the existing data is preserved correctly.

```
void resize(Cart & c, int s=5);
```

```
void resize(Cart &c, int s = 5)
{
    // New capacity
    int newCapacity = c.capacity + s;

    // Allocate new array with increased capacity
    Item* newData = new Item[newCapacity];

    // Copy existing items into new array
    for (int i = 0; i < c.noOfItems; i++)
    {
        newData[i] = c.data[i];
    }

    // Deallocate old memory
    delete[] c.data;

    // Update cart to point to new array
    c.data = newData;
    c.capacity = newCapacity;
}
```